AWS Cloud Portfolio

Serverless Contact Management System

A real-world cloud project demonstrating Lambda · DynamoDB · API Gateway SES · S3 · IAM · CloudWatch · Secrets Manager · CI/CD

12+	$0	100%	2
AWS services integrated	Monthly cost (free tier)	Serverless architecture	Lambda functions

01

Background & Context

This project was built to create a tangible, deployable demonstration of AWS cloud skills — not a tutorial reproduction, but a system designed from first principles with intentional architectural choices documented throughout.

Skills this project draws on:

- **ALX Africa AWS Cloud Computing Programme** — Solutions architecture, serverless, storage, databases, security, CI/CD
- **Microsoft Azure** — AZ-900 (Fundamentals) and DP-900 (Data Fundamentals) — multi-cloud perspective applied to AWS IAM and data layer decisions
- **Dynamic Database** — Professional cloud environment, Azure in production, real operational pressure
- **UI/UX Design** — Startup experience in French market, design systems, user-centred thinking — informs how APIs and interfaces are structured
- **Information Systems** — Web development, web design, systems thinking — underpins the full-stack understanding

02

Problem Statement

Design and deploy a production-grade backend system on AWS that:

- Accepts contact form submissions from any frontend
- Persists data reliably to a managed database
- Sends transactional email confirmations to submitters

- Costs nothing to run at low-to-medium traffic volumes
- Deploys automatically on every code change
- Is fully observable via logging and monitoring
- Follows AWS security best practices from day one

03

System Architecture

The system follows a fully serverless, event-driven architecture. There are no servers to provision, patch, or scale — AWS manages all compute infrastructure.

```
User / Browser
|
v
Amazon API Gateway (HTTP API — CORS configured)
|
+---- POST /contact ------> Lambda: contact_form
| |
| +---> DynamoDB (persist submission)
| +---> SES (email confirmation)
| +---> CloudWatch (execution logs)
|
+---- GET /submissions ---> Lambda: get_submissions
|
+---> DynamoDB (scan all records)
GitHub push to main
|
v
GitHub Actions CI/CD
|
+---> Lambda: auto-package and deploy both functions
S3 Bucket <-- Static assets (architecture diagrams, screenshots)
```

04

AWS Services — What Each One Does

Service	Role	Free Tier
Lambda	Serverless compute. Two functions: contact handler and submission reader. No server management, pay-per	1M requests/month free
API Gateway	HTTP API layer. Exposes Lambda functions as REST endpoints with CORS headers, routing, and request va	1M calls/month free
DynamoDB	NoSQL database storing every contact submission. PAY_PER_REQUEST billing — zero cost at zero traffic,	25GB + 25 WCU free
SES	Simple Email Service. Sends transactional confirmation emails to form submitters. Requires verified sender id	62K emails/month free

Service	Role	Free Tier
IAM	Identity and Access Management. Lambda execution role scoped to minimum required permissions — Dynar	Always free
CloudWatch	Automatic Lambda execution logging. 30-day log retention policy set on log group.	5GB ingestion invocations, errors, and cus
S3	Object storage for static assets — architecture diagrams, documentation, screenshots, supporting files.	5GB free
Secrets Manager	Secure storage for any API keys or sensitive configuration. Referenced at runtime — no credentials in code.	30-day trial
GitHub Actions	CI/CD pipeline. On every push to main: packages Lambda code into ZIP, deploys to AWS automatically. No r	Free for public repos

05

Key Architectural Decisions

Decision	Chosen	Reason
Compute	Lambda vs EC2	EC2 runs 24/7 at cost. Lambda runs only on invocation — $0 at zero traffic, automatic s
Database	DynamoDB vs RDS	Contact submissions are schema-simple (name, email, message, timestamp). DynamoD
Auth	IAM roles vs keys	Lambda execution roles attach directly to the function runtime. No credentials in code, n
API layer	API GW vs ALB	API Gateway HTTP API is lower cost and simpler for Lambda proxy integrations at this s
CI/CD	GitHub Actions vs CodePipeline	GitHub Actions is free for public repositories and integrates directly with AWS CLI. Code

06

AWS Well-Architected Framework Alignment

Operational Excellence

CI/CD pipeline auto-deploys on every commit. CloudWatch logs capture all executions. 30-day log retention prevents unbounded storage cost.

Security

IAM roles scoped to minimum permissions. No hardcoded credentials anywhere in the codebase. Secrets Manager used for sensitive values. HTTPS enforced by API Gateway.

Reliability

Lambda and DynamoDB are both managed, multi-AZ services — AWS handles availability. PAY_PER_REQUEST DynamoDB auto-scales with traffic spikes.

Performance Efficiency

Serverless architecture — no idle compute. Lambda cold-starts are minimal for Python at 128MB memory. DynamoDB single-digit millisecond reads.

Cost Optimisation

Zero cost at zero traffic. Every service operates within AWS Free Tier. PAY_PER_REQUEST pricing on all usage-based services.

Sustainability

No always-on infrastructure. Resources provisioned only when requests arrive. Minimal energy footprint for this workload.

07

End-to-End Flow Walkthrough

1. Request arrives

A user submits the contact form. The browser sends a POST request to the API Gateway endpoint with JSON body: {name, email, message}.

2. API Gateway routes

API Gateway validates the HTTP method, applies CORS headers, and proxies the request payload to the contact_form Lambda function.

3. Lambda validates

Lambda parses the JSON body, checks all required fields are present, and returns a 400 error with a descriptive message if any are missing.

4. DynamoDB write

A UUID is generated as the partition key. The submission (name, email, message, timestamp, status) is written to the portfolio-contacts table using PAY_PER_REQUEST billing.

5. SES email

Lambda calls SES to send a confirmation email to the submitter's address. This is non-fatal — if SES fails, the DynamoDB write is already committed and the error is logged.

6. CloudWatch log

Every step emits structured log entries to CloudWatch. Lambda's execution role grants PutLogEvents automatically.

7. Response

Lambda returns a 200 JSON response with the submission ID. API Gateway adds CORS headers and returns it to the browser.

08

Reflections & What This Demonstrates

Building this project required making real engineering decisions — not following a guided lab. The choices above had to be reasoned through, not selected from a dropdown.

What this project shows:

- Ability to design a multi-service AWS architecture from a requirement, not a tutorial
- Understanding of serverless trade-offs vs traditional compute
- IAM security thinking — least privilege, role-based access, credential-free architecture
- Cost awareness — every service choice has a cost justification
- CI/CD discipline — infrastructure as code, automated deployment pipeline
- AWS Well-Architected Framework applied at the design stage, not retrofitted
- Multi-cloud context — Azure background informs cross-platform perspective
- UI/UX background — API design is user-centred; error messages are clear and actionable

---